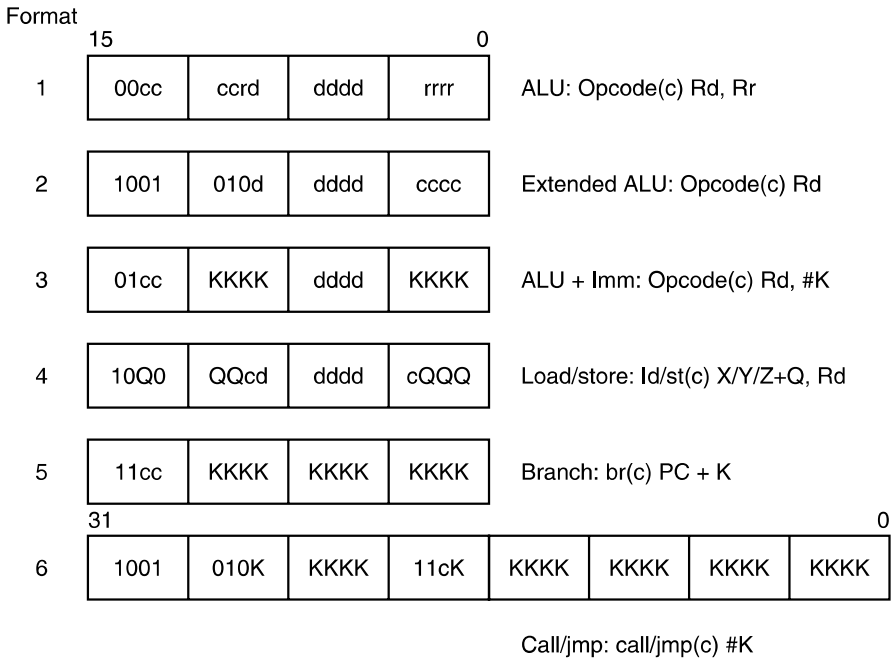

Figure 5-15. The ATmega168 AVR instruction formats.

number of opcode bits is cut in half, allowing only four instructions to use this format (SUBCI, SUBI, ORI, and ANDI).

Format 4 encodes load and store instruction, which includes a 6-bit unsigned immediate operand. The base register is a fixed register not specified in the instruction encoding because it is implied by the load/store opcode.

Formats 5 and 6 are used for jumps and procedure calls. The first format includes a 12-bit signed immediate value that is added to the instruction's PC value to compute the target of the instruction. The last format expands the offset to 22 bits, by making the AVR instruction 32 bits in size.

5.4 ADDRESSING

Most instructions have operands, so some way is needed to specify where they are. This subject, which we will now discuss, is called **addressing**.

5.4.1 Addressing Modes

Up until now, we have paid little attention to how the bits of an address field are interpreted to find the operand. It is now time to investigate this subject, called **address modes**. As we shall see, there are many ways to do it.

5.4.2 Immediate Addressing

The simplest way for an instruction to specify an operand is for the address part of the instruction actually to contain the operand itself rather than an address or other information describing where the operand is. Such an operand is called an **immediate operand** because it is automatically fetched from memory at the same time the instruction itself is fetched; hence it is immediately available for use. A possible immediate instruction for loading register R1 with the constant 4 is shown in Fig. 5-16.

Figure 5-16. An immediate instruction for loading 4 into register 1.

Immediate addressing has the virtue of not requiring an extra memory reference to fetch the operand. It has the disadvantage that only a constant can be supplied this way. Also, the number of values is limited by the size of the field. Still, many architectures use this technique for specifying small integer constants.

5.4.3 Direct Addressing

A method for specifying an operand in memory is just to give its full address. This mode is called **direct addressing**. Like immediate addressing, direct addressing is restricted in its use: the instruction will always access exactly the same memory location. So while the value can change, the location cannot. Thus direct addressing can only be used to access global variables whose address is known at compile time. Nevertheless, many programs have global variables, so this mode is widely used. The details of how the computer knows which addresses are immediate and which are direct will be discussed later.

5.4.4 Register Addressing

Register addressing is conceptually the same as direct addressing but specifies a register instead of a memory location. Because registers are so important (due to fast access and short addresses) this addressing mode is the most common one on most computers. Many compilers go to great lengths to determine which variables will be accessed most often (for example, a loop index) and put these variables in registers.

This addressing mode is known simply as **register mode**. In load/store architectures such as the OMAP4420 ARM architecture, nearly all instructions use this addressing mode exclusively. The only time this addressing mode is not used is when an operand is moved from memory into a register (LDR instruction) or from a register to memory (STR instruction). Even for those instructions, one of the operands is a register—where the memory word is to come from or go to.

5.4.5 Register Indirect Addressing

In this mode, the operand being specified comes from memory or goes to memory, but its address is not hardwired into the instruction, as in direct addressing. Instead, the address is contained in a register. An address used in this manner is called a **pointer**. A big advantage of register indirect addressing is that it can reference memory without paying the price of having a full memory address in the instruction. It can also use different memory words on different executions of the instruction.

To see why using a different word on each execution might be useful, imagine a loop that steps through the elements of a 1024-element one-dimensional integer array to compute the sum of the elements in register R1. Outside the loop, some other register, say, R2, can be set to point to the first element of the array, and another register, say, R3, can be set to point to the first address beyond the array. With 1024 integers of 4 bytes each, if the array begins at A, the first address beyond the array will be $A + 4096$. Typical assembly code for doing this calculation is shown in Fig. 5-17 for a two-address machine.

```
        MOV R1,#0           ; accumulate the sum in R1, initially 0
        MOV R2,#A           ; R2 = address of the array A
        MOV R3,#A+4096      ; R3 = address of the first word beyond A
LOOP:   ADD R1,(R2)         ; register indirect through R2 to get operand
        ADD R2,#4           ; increment R2 by one word (4 bytes)
        CMP R2,R3           ; are we done yet?
        BLT LOOP            ; if R2 < R3, we are not done, so continue
```

Figure 5-17. A generic assembly program for computing the sum of the elements of an array.

In this little program, we use several addressing modes. The first three instructions use register mode for the first operand (the destination) and immediate mode for the second operand (a constant indicated by the # sign). The second instruction puts the *address* of A in R2, not the contents. That is what the # sign tells the assembler. Similarly, the third instruction puts the address of the first word beyond the array in R3.

What is interesting to note is that the body of the loop itself does not contain any memory addresses. It uses register and register indirect mode in the fourth instruction. It uses register and immediate mode in the fifth instruction and register mode twice in the sixth instruction. The BLT might use a memory address, but more likely it specifies the address to branch to with an 8-bit offset relative to the BLT instruction itself. By avoiding the use of memory addresses completely, we have produced a short, fast loop. As an aside, this program is really for the Core i7, except that we have renamed the instructions and registers and changed the

notation to make it easy to read because the Core i7's standard assembly-language syntax (MASM) verges on the bizarre, a remnant of the machine's former life as an 8088.

It is worth noting that, in theory, there is another way to do this computation without using register indirect addressing. The loop could have contained an instruction to add *A* to R1, such as

```
ADD R1,A
```

Then on each iteration of the loop, the instruction itself could be incremented by 4, so that after one iteration it read

```
ADD R1,A+4
```

and so on until it was done.

A program that modifies itself like this is called a **self-modifying** program. The idea was thought of by none other than John von Neumann and made sense on early computers, which did not have register indirect addressing. Nowadays, self-modifying programs are considered horrible style and hard to understand. They also cannot be shared among multiple processes at the same time. Furthermore, they will not even work correctly on machines with a split level 1 cache if the I-cache has no circuitry for doing writebacks (because the designers assumed that programs do not modify themselves). Lastly, self-modifying programs will also fail on machines with separate instruction and data spaces. All in all, this is an idea that has come and (fortunately) gone.

5.4.6 Indexed Addressing

It is frequently useful to be able to reference memory words at a known offset from a register. We saw some examples with IJVM where local variables are referenced by giving their offset from LV. Addressing memory by giving a register (explicit or implicit) plus a constant offset is called **indexed addressing**.

Local variable access in IJVM uses a pointer into memory (LV) in a register plus a small offset in the instruction itself, as shown in Fig. 4-19(a). However, it is also possible to do it the other way: the memory pointer in the instruction and the small offset in the register. To see how that works, consider the following calculation. We have two one-dimensional arrays of 1024 words each, *A* and *B*, and we wish to compute A_i AND B_i for all the pairs and then OR these 1024 Boolean products together to see if there is at least one nonzero pair in the set. One approach would be to put the address of *A* in one register, the address of *B* in a second register, and then step through them together in lockstep, analogous to what we did in Fig. 5-17. This way of doing it would certainly work, but it can be done in a better, more general way, as illustrated in Fig. 5-18.

```
        MOV R1,#0          ; accumulate the OR in R1, initially 0
        MOV R2,#0          ; R2 = index, i, of current product: A[i] AND B[i]
        MOV R3,#4096       ; R3 = first index value not to use
LOOP:   MOV R4,A(R2)       ; R4 = A[i]
        AND R4,B(R2)       ; R4 = A[i] AND B[i]
        OR R1,R4           ; OR all the Boolean products into R1
        ADD R2,#4          ; i = i + 4 (step in units of 1 word = 4 bytes)
        CMP R2,R3          ; are we done yet?
        BLT LOOP           ; if R2 < R3, we are not done, so continue
```

Figure 5-18. A generic assembly program for computing the OR of A_i AND B_i for two 1024-element arrays.

Operation of this program is straightforward. We need four registers here:

1. R1 — Holds the accumulated OR of the Boolean product terms.
2. R2 — The index, i, that is used to step through the arrays.
3. R3 — The constant 4096, which is the lowest value of i not to use.
4. R4 — A scratch register for holding each product as it is formed.

After initializing the registers, we enter the six-instruction loop. The instruction at *LOOP* fetches A_i into R4. The calculation of the source here uses indexed mode. A register, R2, and a constant, the address of A, are added together and used to reference memory. The sum of these two quantities goes to the memory but is not stored in any user-visible register. The notation

```
MOV R4,A(R2)
```

means that the destination uses register mode with R4 as the register and the source uses indexed mode, with A as the offset and R2 as the register. If A has the value, say, 124300, the actual machine instruction for this is likely to look something like the one shown in Fig. 5-19.

MOV	R4	R2	124300

Figure 5-19. A possible representation of MOV R4,A(R2).

The first time through the loop, R2 is 0 (due to it being initialized that way), so the memory word addressed is A_0, at address 124300. This word is loaded into R4. The next time though the loop, R2 is 4, so the memory word addressed is A_1, at 124304, and so on.

As we promised earlier, here the offset in the instruction itself is the memory pointer and the value in the register is a small integer that is incremented during the

calculation. This form requires an offset field in the instruction large enough to hold an address, of course, so it is less efficient than doing it the other way; however, it is nevertheless frequently the best way.

5.4.7 Based-Indexed Addressing

Some machines have an addressing mode in which the memory address is computed by adding up two registers plus an (optional) offset. Sometimes this mode is called **based-indexed addressing**. One of the registers is the base and the other is the index. Such a mode would have been useful here. Outside the loop we could have put the address of *A* in R5 and the address of *B* in R6. Then we could have replaced the instruction at *LOOP* and its successor with

```
LOOP:   MOV R4,(R2+R5)
        AND R4,(R2+R6)
```

If there were an addressing mode for indirecting through the sum of two registers with no offset, that would be ideal. Alternatively, even an instruction with an 8-bit offset would have been an improvement over the original code since we could set both offsets to 0. If, however, the offsets are always 32 bits, then we have not gained anything by using this mode. In practice, however, machines that have this mode usually have a form with an 8-bit or 16-bit offset.

5.4.8 Stack Addressing

We have already noted that making machine instructions as short as possible is highly desirable. The ultimate limit in reducing address lengths is having no addresses at all. As we saw in Chap. 4, zero-address instructions, such as IADD, are possible in conjunction with a stack. In this section we will look at stack addressing more closely.

Reverse Polish Notation

It is an ancient tradition in mathematics to put the operator between the operands, as in $x + y$, rather than after the operands, as in $x\ y +$. The form with the operator "in" between the operands is called **infix** notation. The form with the operator after the operands is called **postfix** or **reverse Polish notation**, after the Polish logician J. Lukasiewicz (1958), who studied the properties of this notation.

Reverse Polish notation has a number of advantages over infix for expressing algebraic formulas. First, any formula can be expressed without parentheses. Second, it is convenient for evaluating formulas on computers with stacks. Third, infix operators have precedence, which is arbitrary and undesirable. For example, we

know that $a \times b + c$ means $(a \times b) + c$ and not $a \times (b + c)$ because multiplication has been arbitrarily defined to have precedence over addition. But does left shift have precedence over Boolean AND? Who knows? Reverse Polish notation eliminates this nuisance.

Several algorithms for converting infix formulas into reverse Polish notation exist. The one given below is an adaptation of an idea due to E. W. Dijkstra. Assume that a formula is composed of the following symbols: variables, the dyadic (two-operand) operators + − * /, and left and right parentheses. To mark the ends of a formula, we will insert the symbol ⊥ after the last symbol and before the first symbol.

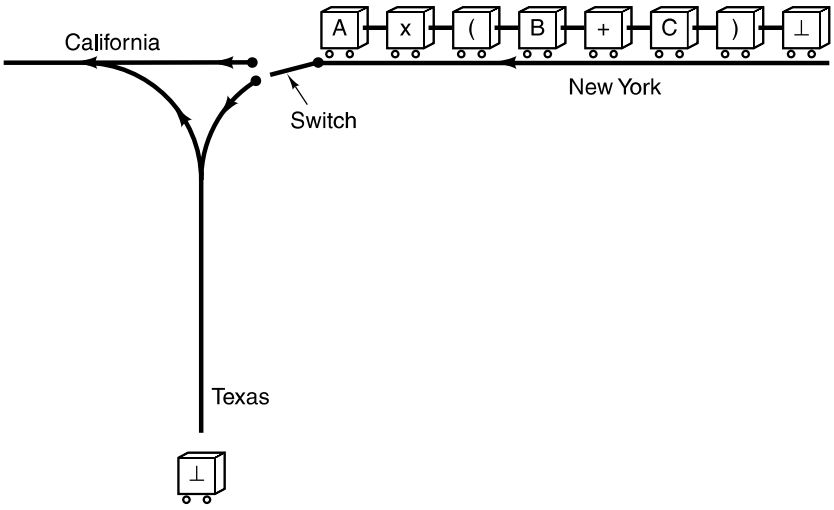

Figure 5-20. Each railroad car represents one symbol in the formula to be converted from infix to reverse Polish notation.

Figure 5-20 shows a railroad track from New York to California, with a spur in the middle that heads off in the direction of Texas. Think of the New York to California line as the main line with the branch down to Texas as a siding for temporary storage. The names and directions are not important. What matters is the distinction between the main line and the alternative place for temporarily storing things. Each symbol in the formula is represented by one railroad car. The train moves westward (to the left). When each car arrives at the switch, it must stop just before it and ask if it should go to California directly or take a side trip to Texas. Cars containing variables always go directly to California and never to Texas. Cars containing all other symbols must inquire about the contents of the nearest car on the Texas line before entering the switch.

The table of Fig. 5-21 shows what happens, depending on the contents of the nearest car on the Texas line and the car poised at the switch. The first ⊥ always goes to Texas. The numbers refer to the following situations:

1. The car at the switch heads toward Texas.
2. The most recent car on the Texas line turns and goes to California.
3. Both the car at the switch and the most recent car on the Texas line are diverted and disappear (i.e., both are deleted).
4. Stop. The symbols now in California represent the reverse Polish notation formula when read from left to right.
5. Stop. An error has occurred. The original formula was not correctly balanced.

Most recently arrived car on the Texas line	Car at the switch: ⊥	+	−	x	/	(	)
⊥	4	1	1	1	1	1	5
+	2	2	2	1	1	1	2
−	2	2	2	1	1	1	2
x	2	2	2	2	2	1	2
/	2	2	2	2	2	1	2
(	5	1	1	1	1	1	3

Figure 5-21. Decision table used by the infix-to-reverse Polish notation algorithm

After each action is taken, a new comparison is made between the car currently at the switch, which may be the same car as in the previous comparison or may be the next car, and the car that is now the last one on the Texas line. The process continues until step 4 is reached. Notice that the Texas line is being used as a stack, with routing a car to Texas being a push operation, and turning a car already on the Texas line around and sending it to California being a pop operation.

Infix	Reverse Polish notation
A + B × C	A B C × +
A × B + C	A B × C +
A × B + C × D	A B × C D × +
(A + B) / (C – D)	A B + C D – /
A × B / C	A B × C /
((A + B) × C + D)/(E + F + G)	A B + C × D + E F + G + /

Figure 5-22. Some examples of infix expressions and their reverse Polish notation equivalents.

The order of the variables is the same in infix and in reverse Polish notation. The order of the operators, however, is not always the same. Operators appear in

reverse Polish notation in the order they will actually be executed during the evaluation of the expression. Figure 5-22 gives several examples of infix formulas and their reverse Polish notation equivalents.

Evaluation of Reverse Polish Notation Formulas

Reverse Polish notation is the ideal notation for evaluating formulas on a computer with a stack. The formula consists of n symbols, each one either an operand or an operator. The algorithm for evaluating a reverse Polish notation formula using a stack is simple. Scan the reverse Polish notation string from left to right. When an operand is encountered, push it onto the stack. When an operator is encountered, execute the corresponding instruction.

Figure 5-23 shows the evaluation of

$$(8 + 2 \times 5) / (1 + 3 \times 2 - 4)$$

in IJVM. The corresponding reverse Polish notation formula is

$$8\ 2\ 5 \times +\ 1\ 3\ 2 \times +\ 4 -\ /$$

In the figure, we have introduced IMUL and IDIV as the multiplication and division instructions, respectively. The number on top of the stack is the right operand, not the left operand. This point is important for division (and subtraction) since the order of the operands is significant (unlike addition and multiplication). In other words, IDIV has been carefully defined so that first pushing the numerator, then pushing the denominator, and then doing the operation gives the correct result. Notice how easy code generation is from reverse Polish notation to IJVM: just scan the reverse Polish notation formula and output one instruction per symbol. If the symbol is a constant or variable, output an instruction to push it onto the stack. If the symbol is an operator, output an instruction to perform the operation.

5.4.9 Addressing Modes for Branch Instructions

So far we have been looking only at instructions that operate on data. Branch instructions (and procedure calls) also need addressing modes for specifying the target address. The modes we have examined so far also work for branches for the most part. Direct addressing is certainly a possibility, with the target address simply being included in the instruction in full.

However, other addressing modes also make sense. Register indirect addressing allows the program to compute the target address, put it in a register, and then go there. This mode gives the most flexibility since the target address is computed at run time. It also presents the greatest opportunity for creating bugs that are nearly impossible to find.

Another reasonable mode is indexed mode, which offsets a known distance from a register. It has the same properties as register indirect mode.